

Visualisation

July 31, 2023

APPLIED LINGUISTICS GRADUATE PROGRAMME (LAEL)

1 Visualisation Practice Exercise 1

1.1 Python Data Visualization with Seaborn and Matplotlib

This practice exercise was guided by this [tutorial](#). The dataset is related to FIFA 19 players and can be obtained [here](#).

1.1.1 Histogram

```
[1]: # Importing the required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
```

```
[2]: # Casting the FIFA 19 data in to the dataframe 'df'
# Please make sure the 'fifa_eda.csv' is in the folder
df = pd.read_csv('fifa_eda.csv')
print(df.dtypes)
df.head()
```

ID	int64
Name	object
Age	int64
Nationality	object
Overall	int64
Potential	int64
Club	object
Value	float64
Wage	float64
Preferred Foot	object
International Reputation	float64
Skill Moves	float64
Position	object
Joined	int64

```

Contract Valid Until      object
Height                    float64
Weight                    float64
Release Clause            float64
dtype: object

```

```

[2]:      ID      Name  Age Nationality  Overall  Potential  \
0  158023    L. Messi   31   Argentina    94         94
1   20801 Cristiano Ronaldo  33    Portugal    94         94
2  190871    Neymar Jr   26     Brazil    92         93
3  193080      De Gea   27      Spain    91         93
4  192985    K. De Bruyne  27     Belgium    91         92

```

```

      Club      Value  Wage Preferred Foot  \
0    FC Barcelona 110500.0  565.0         Left
1      Juventus   77000.0  405.0         Right
2 Paris Saint-Germain 118500.0  290.0         Right
3 Manchester United   72000.0  260.0         Right
4 Manchester City  102000.0  355.0         Right

```

```

      International Reputation  Skill Moves Position  Joined  \
0                             5.0         4.0      RF    2004
1                             5.0         5.0      ST    2018
2                             5.0         5.0      LW    2017
3                             4.0         1.0      GK    2011
4                             4.0         4.0     RCM    2015

```

```

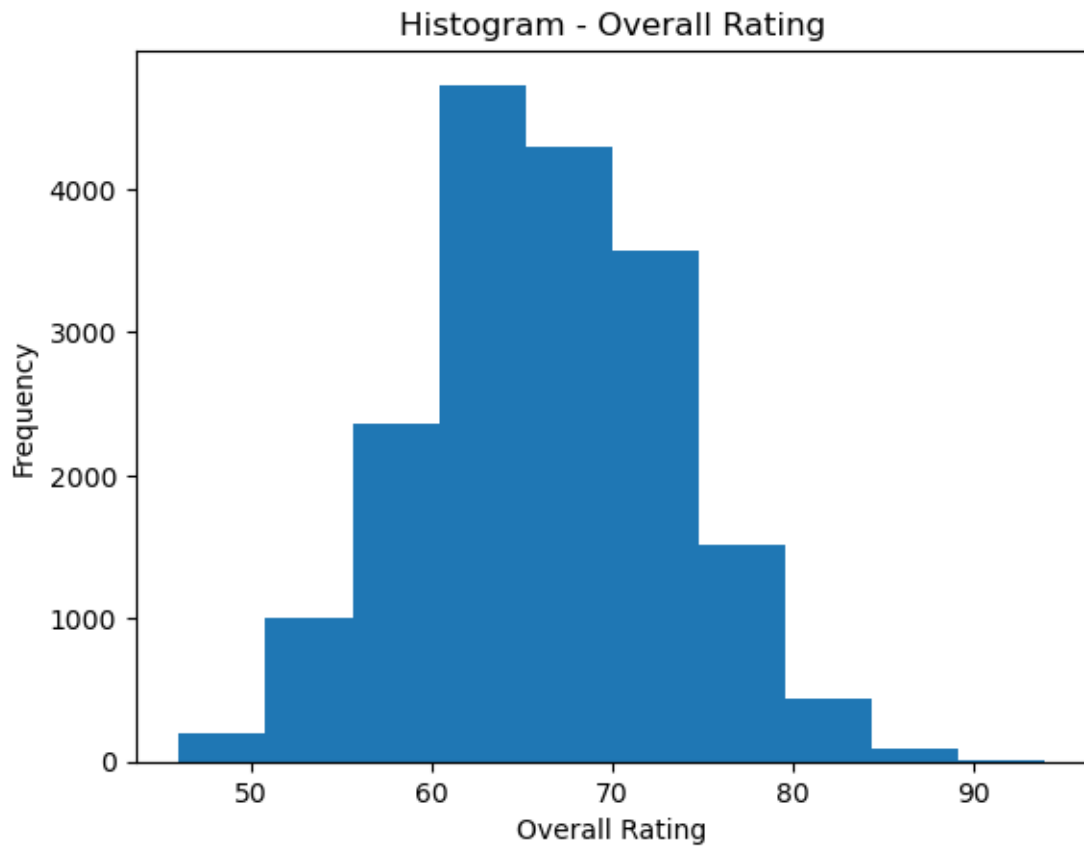
      Contract Valid Until      Height  Weight  Release Clause
0          2021-01-01  5.583333  159.0      226500.0
1          2022-01-01  6.166667  183.0      127100.0
2          2022-01-01  5.750000  150.0      228100.0
3          2020-01-01  6.333333  168.0      138600.0
4          2023-01-01  5.916667  154.0      196400.0

```

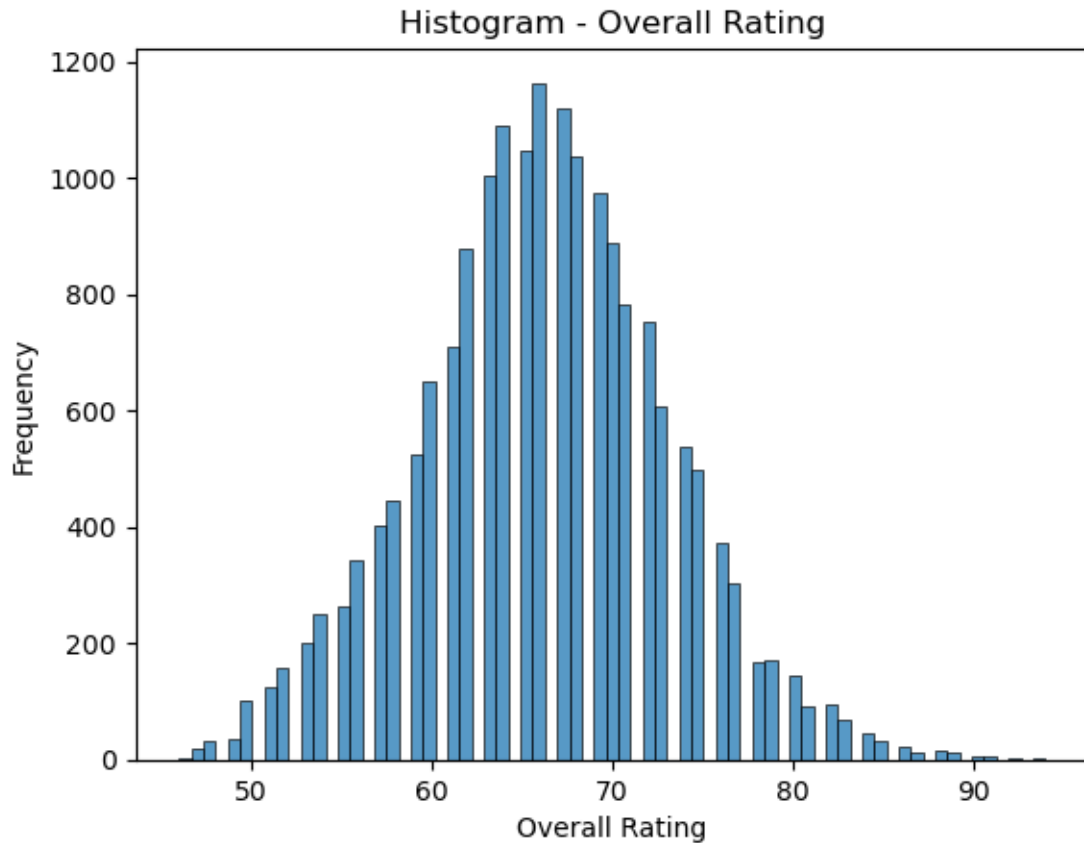
```

[3]: # Histogram of the 'Overall Rating' using Matplotlib
plt.hist(df[['Overall']])
plt.title('Histogram - Overall Rating')
plt.xlabel('Overall Rating')
plt.ylabel('Frequency')
plt.show()

```



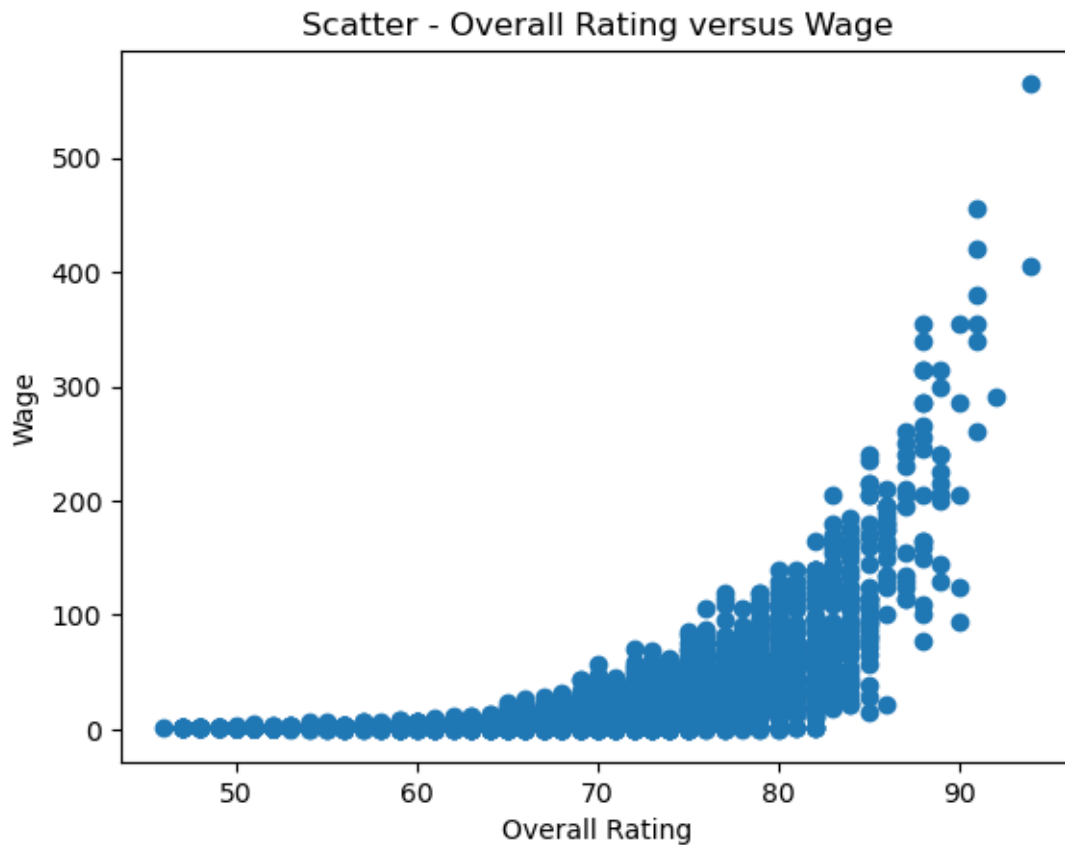
```
[4]: # Histogram of the 'Overall Rating' using Seaborn
sns.histplot(data = df, x = 'Overall')
plt.title('Histogram - Overall Rating')
plt.xlabel('Overall Rating')
plt.ylabel('Frequency')
plt.show()
```



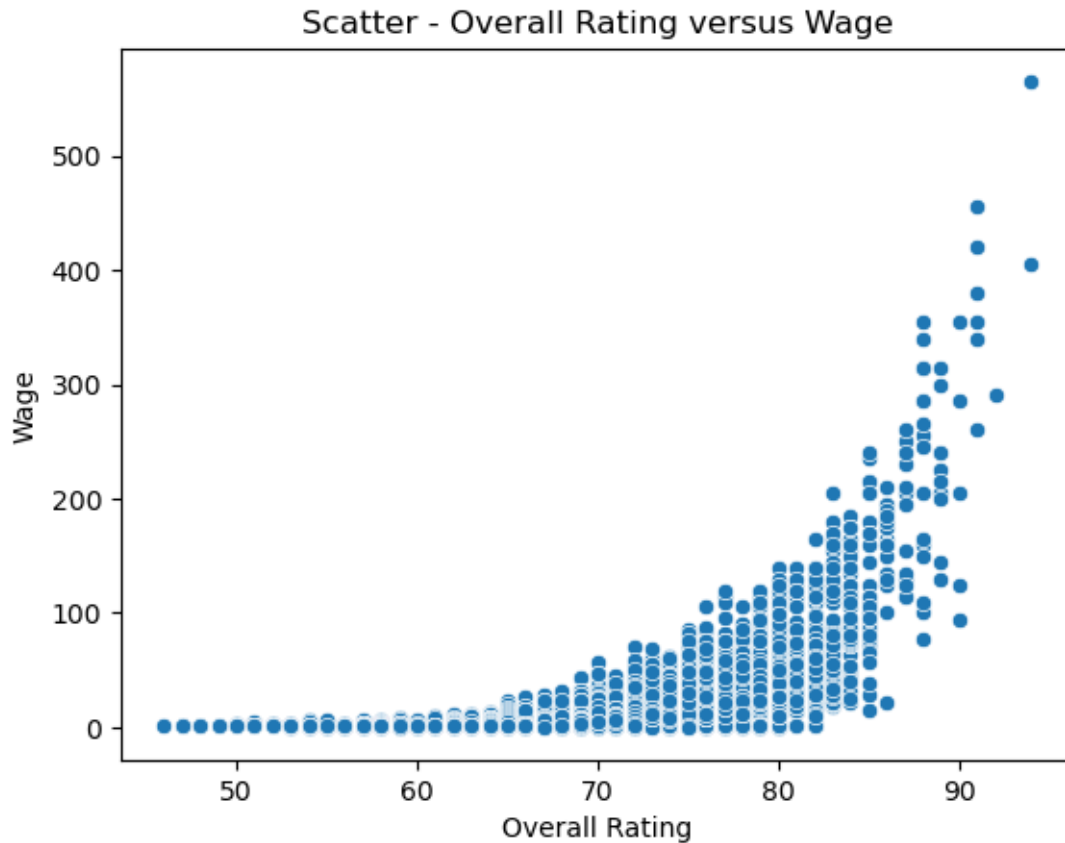
1.1.2 Scatter Plot

```
[5]: # Scatter Plot of 'Overall' versus 'Wage' using Matplotlib
print(df[['Overall', 'Wage']].head())
plt.scatter(df[['Overall']], df[['Wage']])
plt.title('Scatter - Overall Rating versus Wage')
plt.xlabel('Overall Rating')
plt.ylabel('Wage')
plt.show()
```

	Overall	Wage
0	94	565.0
1	94	405.0
2	92	290.0
3	91	260.0
4	91	355.0



```
[6]: # Scatter Plot of 'Overall' versus 'Wage' using Seaborn
sns.scatterplot(data = df, x = 'Overall', y = 'Wage')
plt.title('Scatter - Overall Rating versus Wage')
plt.xlabel('Overall Rating')
plt.ylabel('Wage')
plt.show()
```



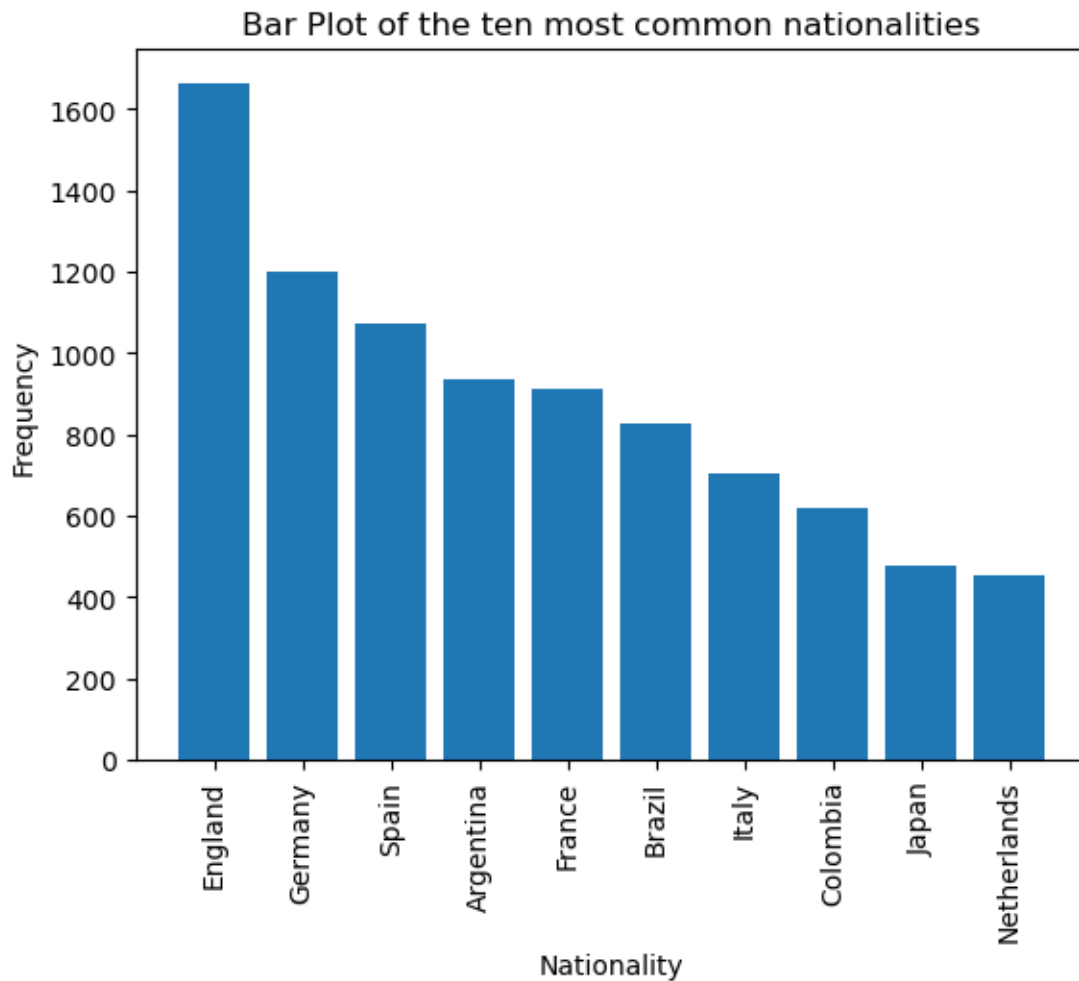
1.1.3 Bar Plot

```
[7]: # Determining the quantity of each nationality
# Notice that 'Counter' requires a 'Series' instead of a 'Dataframe'
from collections import Counter
print(type(df['Nationality']))
print(dict(Counter(df['Nationality']).most_common(10)))
```

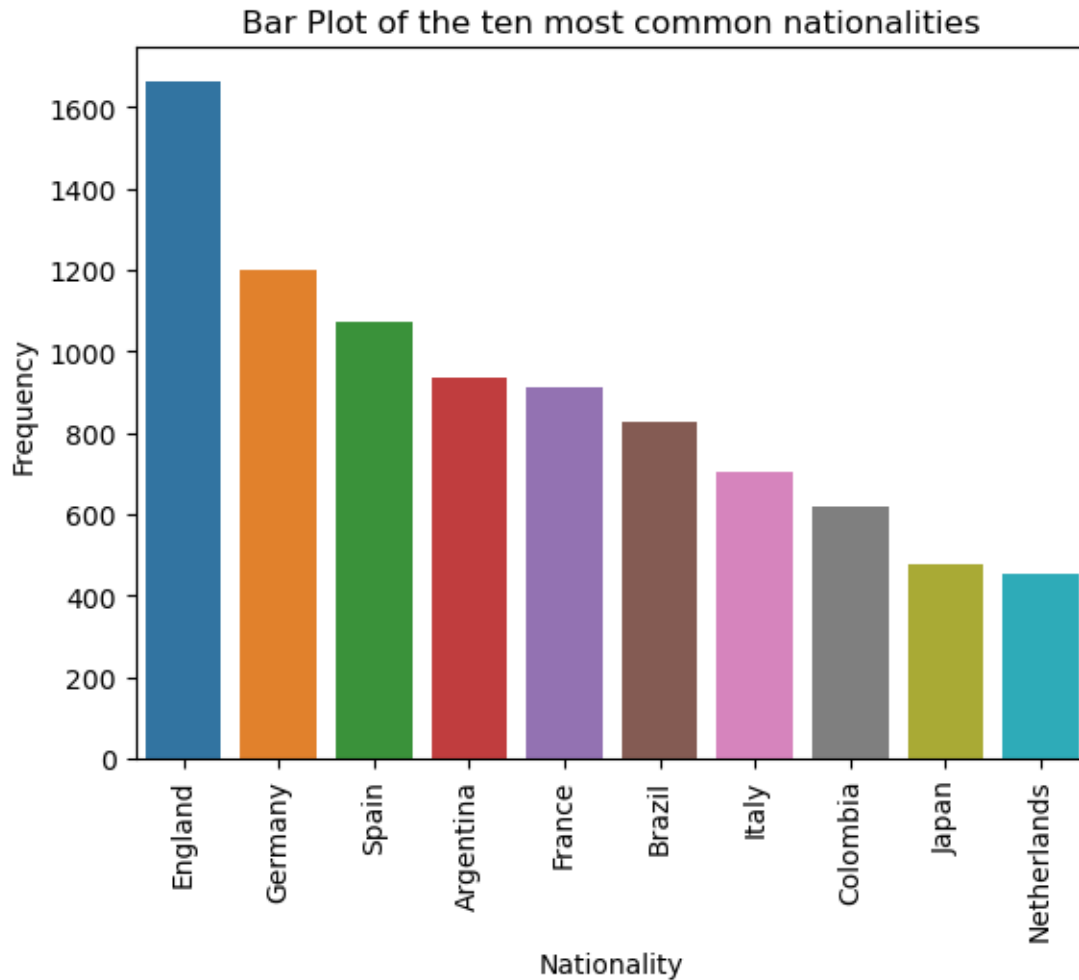
```
<class 'pandas.core.series.Series'>
{'England': 1662, 'Germany': 1198, 'Spain': 1072, 'Argentina': 937, 'France':
914, 'Brazil': 827, 'Italy': 702, 'Colombia': 618, 'Japan': 478, 'Netherlands':
453}
```

```
[8]: # Bar Plot of the top 10 nationalities using Matplotlib
nationality_top10 = dict(Counter(df['Nationality']).most_common(10))
plt.bar(nationality_top10.keys(), nationality_top10.values())
plt.title('Bar Plot of the ten most common nationalities')
plt.xlabel('Nationality')
plt.xticks(rotation = 90)
plt.ylabel('Frequency')
```

```
plt.show()
```



```
[9]: # Bar Plot of the top 10 nationalities using Seaborn
nationality_top10 = dict(Counter(df['Nationality']).most_common(10))
df_nationality_top10 = pd.DataFrame([nationality_top10])
df_nationality_top10
sns.barplot(data = df_nationality_top10)
plt.title('Bar Plot of the ten most common nationalities')
plt.xlabel('Nationality')
plt.xticks(rotation = 90)
plt.ylabel('Frequency')
plt.show()
```



1.1.4 Pie Chart

```
[10]: # Creating a new column 'Main Nationalities' indicating the main nationalities
# In case an unwanted column is added, it can be removed as follows
#df.drop(columns = ['main_nationalities'], inplace = True)
df.loc[df['Nationality'] == 'England', 'Main Nationalities'] = 'England'
df.loc[df['Nationality'] == 'Germany', 'Main Nationalities'] = 'Germany'
df.loc[df['Nationality'] == 'Spain', 'Main Nationalities'] = 'Spain'
df.loc[df['Nationality'] == 'Argentina', 'Main Nationalities'] = 'Argentina'
df.loc[df['Nationality'] == 'France', 'Main Nationalities'] = 'France'
df.loc[df['Nationality'] == 'Brazil', 'Main Nationalities'] = 'Brazil'
df.loc[~df['Nationality'].isin(['England', 'Germany', 'Spain', 'Argentina',
↪ 'France', 'Brazil']), 'Main Nationalities'] = 'Other'
df.head()
```



```
[10]:
```

	ID	Name	Age	Nationality	Overall	Potential	\
0	158023	L. Messi	31	Argentina	94	94	
1	20801	Cristiano Ronaldo	33	Portugal	94	94	
2	190871	Neymar Jr	26	Brazil	92	93	
3	193080	De Gea	27	Spain	91	93	
4	192985	K. De Bruyne	27	Belgium	91	92	

	Club	Value	Wage	Preferred Foot	\
0	FC Barcelona	110500.0	565.0	Left	
1	Juventus	77000.0	405.0	Right	
2	Paris Saint-Germain	118500.0	290.0	Right	
3	Manchester United	72000.0	260.0	Right	
4	Manchester City	102000.0	355.0	Right	

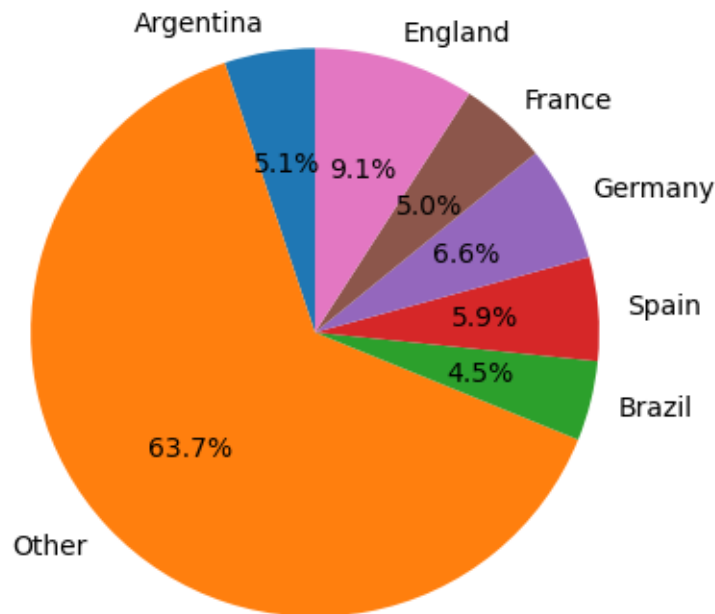
	International Reputation	Skill Moves	Position	Joined	\
0	5.0	4.0	RF	2004	
1	5.0	5.0	ST	2018	
2	5.0	5.0	LW	2017	
3	4.0	1.0	GK	2011	
4	4.0	4.0	RCM	2015	

	Contract Valid Until	Height	Weight	Release Clause	Main Nationalities
0	2021-01-01	5.583333	159.0	226500.0	Argentina
1	2022-01-01	6.166667	183.0	127100.0	Other
2	2022-01-01	5.750000	150.0	228100.0	Brazil
3	2020-01-01	6.333333	168.0	138600.0	Spain
4	2023-01-01	5.916667	154.0	196400.0	Other

```
[11]: # Determining the proportions for the Pie Chart
print(Counter(df['Main Nationalities']))
pie_proportions = dict(Counter(df['Main Nationalities']))
print(pie_proportions)
print(pie_proportions.items())
for key, value in pie_proportions.items():
    pie_proportions[key] = (value/len(df))*100
print(pie_proportions)
```

```
Counter({'Other': 11597, 'England': 1662, 'Germany': 1198, 'Spain': 1072,
'Argentina': 937, 'France': 914, 'Brazil': 827})
{'Argentina': 937, 'Other': 11597, 'Brazil': 827, 'Spain': 1072, 'Germany':
1198, 'France': 914, 'England': 1662}
dict_items([('Argentina', 937), ('Other', 11597), ('Brazil', 827), ('Spain',
1072), ('Germany', 1198), ('France', 914), ('England', 1662)])
{'Argentina': 5.146372274399956, 'Other': 63.69528203438238, 'Brazil':
4.542209040478936, 'Spain': 5.887845334212116, 'Germany': 6.579886856703466,
'France': 5.020047234580106, 'England': 9.12835722524304}
```

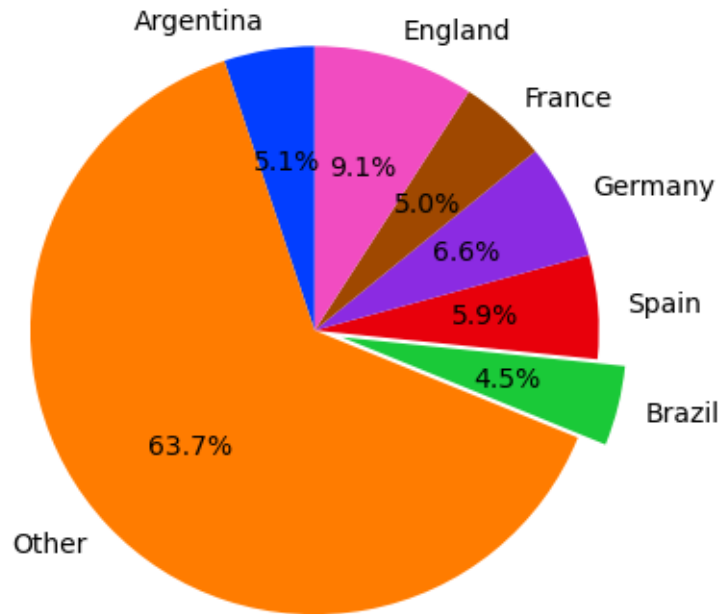
```
[12]: # Pie Chart of the main nationalities using Matplotlib
fig, ax = plt.subplots()
ax.axis('equal')
ax.pie(pie_proportions.values(), labels = pie_proportions.keys(), autopct='%1.1f%%', startangle = 90)
plt.show()
```



```
[13]: # Pie Chart of the main nationalities using Matplotlib and Seaborn
df_pie_proportions = pd.DataFrame([pie_proportions])
print(df_pie_proportions)
df_pie_explode = pd.DataFrame([{'Argentina': 0, 'Other': 0, 'Brazil': 0.1, 'Spain': 0, 'Germany': 0, 'France': 0, 'England': 0}])
print(df_pie_explode)
df_pie_proportions = pd.concat([df_pie_proportions, df_pie_explode], ignore_index = True)
print(df_pie_proportions)
palette = sns.color_palette('bright')
plt.pie(df_pie_proportions.loc[0], labels = list(df_pie_proportions), colors = palette, explode = df_pie_proportions.loc[1], autopct='%1.1f%%', startangle = 90)
plt.show()
```

Argentina Other Brazil Spain Germany France England

0	5.146372	63.695282	4.542209	5.887845	6.579887	5.020047	9.128357
	Argentina	Other	Brazil	Spain	Germany	France	England
0	0	0	0.1	0	0	0	0
	Argentina	Other	Brazil	Spain	Germany	France	England
0	5.146372	63.695282	4.542209	5.887845	6.579887	5.020047	9.128357
1	0.000000	0.000000	0.100000	0.000000	0.000000	0.000000	0.000000

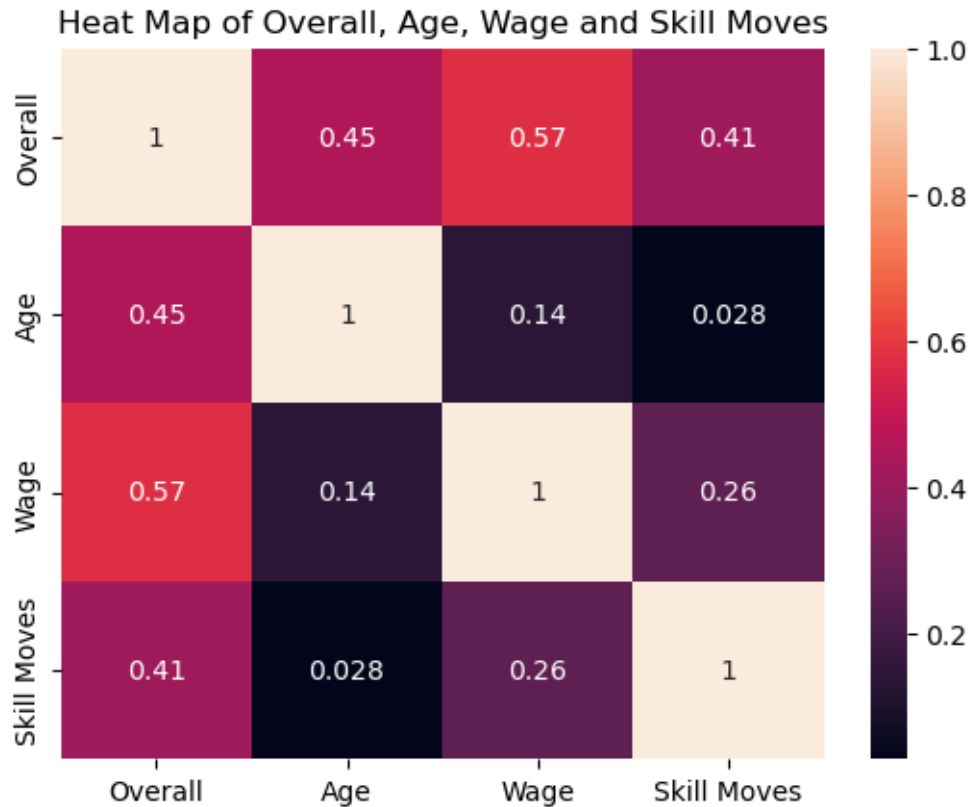


1.1.5 Heat Map

```
[14]: # Heat Map of 'Overall', 'Age', 'Wage', 'Skill Moves' using Seaborn
correlation = df[['Overall', 'Age', 'Wage', 'Skill Moves']].corr()
print(type(correlation))
print(correlation)
sns.heatmap(correlation, annot = True)
plt.title('Heat Map of Overall, Age, Wage and Skill Moves')
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
```

	Overall	Age	Wage	Skill Moves
Overall	1.000000	0.452350	0.571926	0.414463
Age	0.452350	1.000000	0.141145	0.027649
Wage	0.571926	0.141145	1.000000	0.263205
Skill Moves	0.414463	0.027649	0.263205	1.000000



2 Visualisation Practice Exercise 2

2.1 Visualize Distributions With Seaborn

This practice exercise was guided by this [tutorial](#).

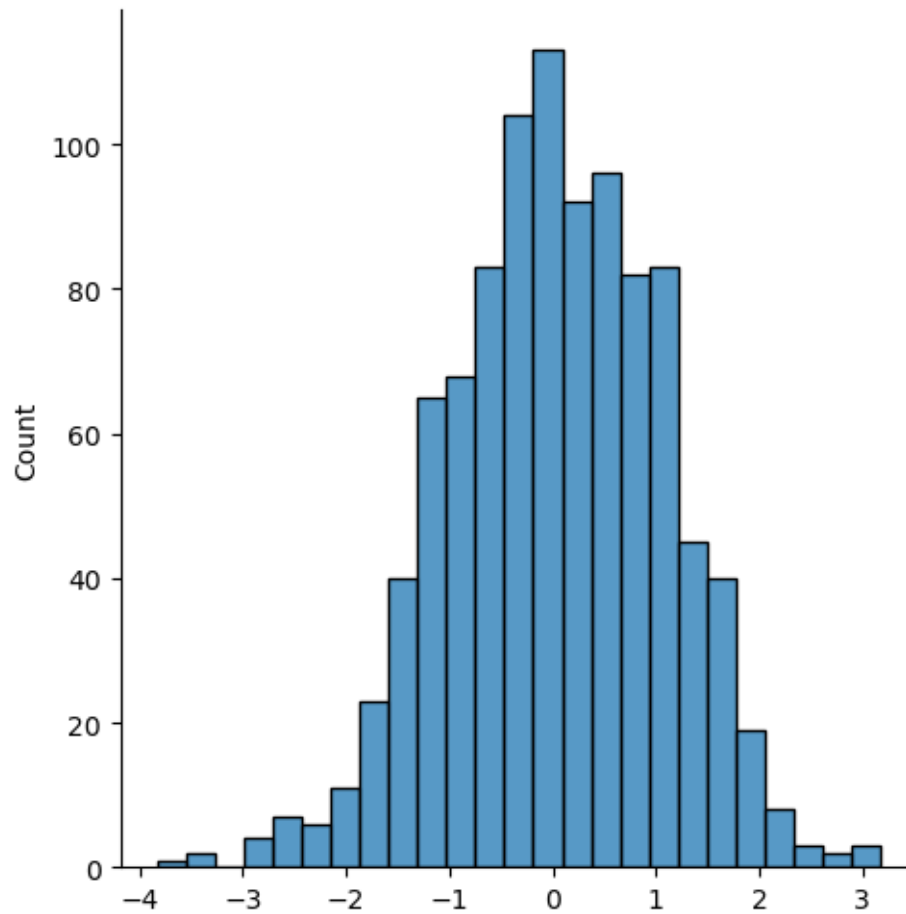
```
[15]: # Importing the required libraries
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import seaborn as sns
```

2.1.1 Normal distribution

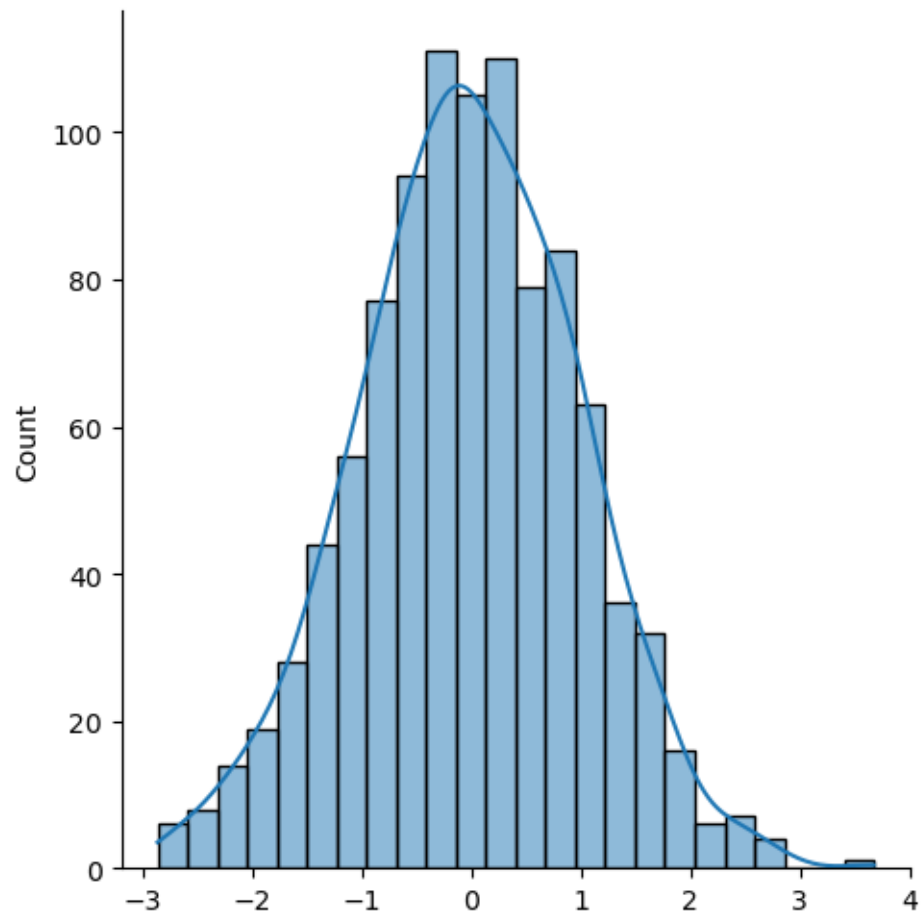
```
[16]: # Trying out Numpy's 'random' function
normal_distribution = np.random.normal(size = (2, 3))
print(normal_distribution)
```

```
[[ 1.3460287 -0.5492698  0.7147615 ]
 [ 0.11518817  0.40616814 -0.87478943]]
```

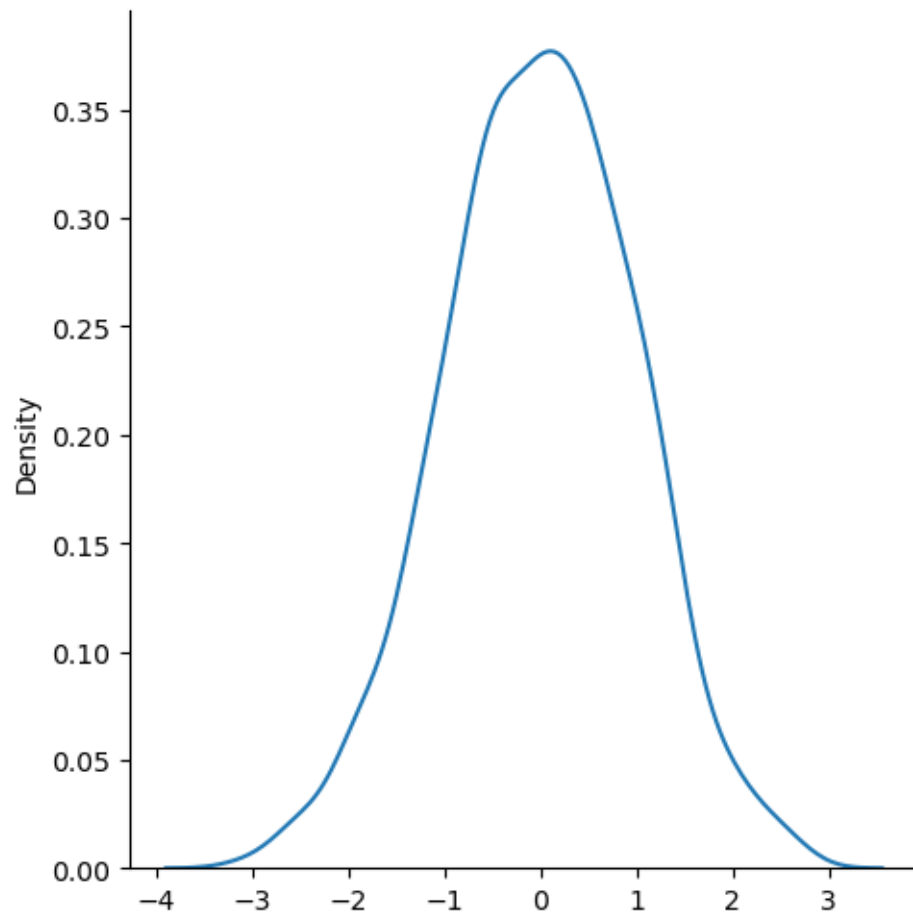
```
[17]: # 'displot' defaults to 'histplot' when the parameter 'kind' is not specified
sns.displot(np.random.normal(size=1000))
plt.show()
```



```
[18]: sns.displot(np.random.normal(size=1000), kde = True)
plt.show()
```



```
[19]: sns.displot(np.random.normal(size=1000), kind = 'kde')  
plt.show()
```



[]: